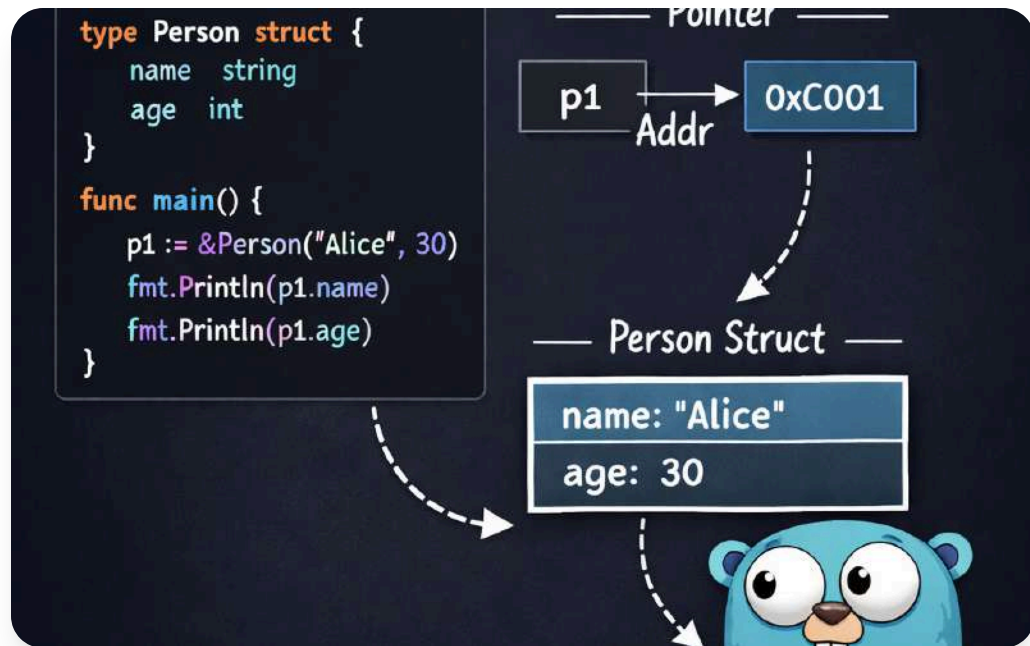




Golang Backend Pertemuan 4

Pointer, struct, dan method Go untuk backend yang efisien dan mudah dirawat.



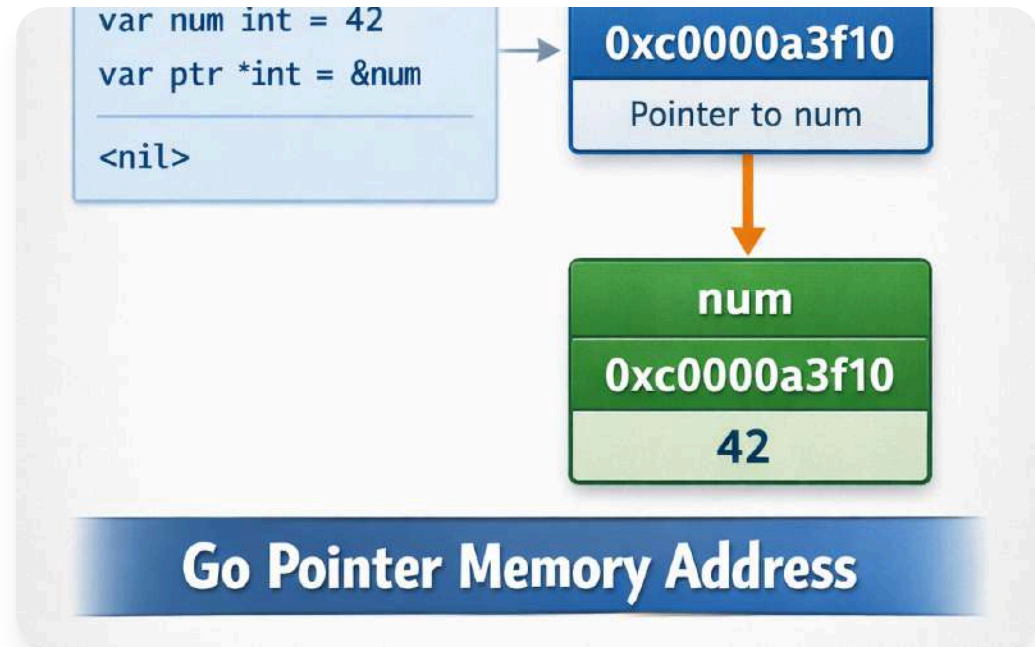
Juara Coding

June 2026



Tujuan Pembelajaran

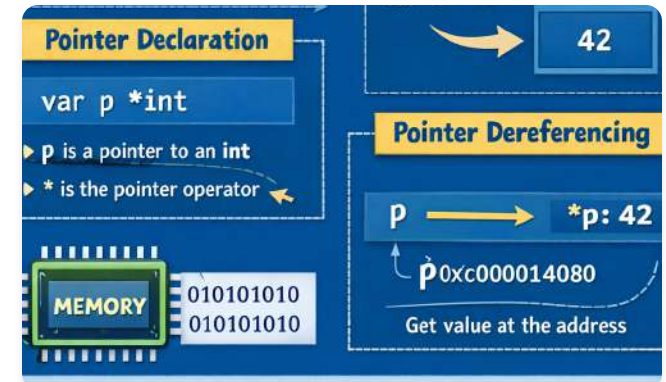
Memahami *pointer*, *method*, *receiver*, dan pola *struct* untuk backend yang rapi.



Review Singkat Pointer di Go

Pointer menyimpan alamat memori suatu nilai sehingga fungsi atau method dapat mengubah data asli secara efisien di backend.

Operator Pointer: & dan *



01 Fungsi operator &

Operator `&` dipakai untuk mengambil alamat memori dari sebuah variabel agar bisa direferensikan.

02 Fungsi operator *

Operator `*` digunakan untuk mengakses nilai dari alamat memori atau melakukan dereference pointer.

03 Penting dalam Go

Pemahaman pointer penting karena banyak penggunaan struct dan method di Go bergantung pada referensi data.

Pointer dan Efisiensi Data

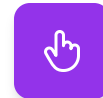


Struct besar yang dikirim sebagai nilai akan disalin penuh. *Pointer* membantu efisiensi pada request, model data, dan dependency backend.



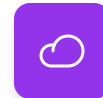
Salinan Struct Besar

Mengirim struct besar sebagai value membuat Go menyalin seluruh isi data ke memori baru.



Gunakan Pointer

Pointer mengurangi salinan data agar proses lebih efisien pada objek yang dipakai ulang.



Relevan di Backend

Berguna saat memproses request, model data, atau dependency yang digunakan berulang.

```
Name      string "json:\"name\""
Email     string "tjson:\"email\""
CreatedAt time.Time "json:\"created_at\""
}

type Post struct {
    ID        int    "json:\"id\""
    UserID    int    "json:\"user_id\""
    Title     string "json:\"title\""
    Content   string "json:\"content\""
    CreatedAt time.Time "json:\"created_at\""
}

type Comment struct {
    ID        int    "json:\"id\""
    PostID    int    "json:\"post_id\""
    UserID    int    "json:\"user_id\""
    Text      string "json:\"text\""
    CreatedAt time.Time "json:\"created_at\""
}
```

Struct untuk Data Backend

Struct memodelkan user, payload, response, konfigurasi, dan service secara rapi serta aman.

Definisi Struct User

Field	Tipe	Fungsi
ID	int	Identitas unik pengguna
Name	string	Menyimpan nama pengguna
Email	string	Menyimpan alamat email
Active	bool	Status aktif pengguna

Struct **User** menunjukkan cara Go memodelkan data backend secara jelas, terstruktur, dan mudah divalidasi untuk pengembangan fitur.

Inisialisasi Struct

```
5 }  
6 type Person struct {  
7     Name string  
8     Age: int  
9     Email string  
10 }  
11  
12 func main() {  
13     p1 := Person({  
14         Name: "Alice",  
15         Age: 30,  
16         Email: "alice@example.com",  
17     })  
18 }  
19 fmt.Println(p1)  
20 }
```

01 *Urutan field*

Struct bisa dibuat berdasarkan urutan deklarasi field, tetapi cara ini lebih rentan salah saat struktur berkembang.

02 *Nama field*

Inisialisasi dengan nama field adalah pilihan paling aman karena lebih jelas, mudah dibaca, dan stabil saat field bertambah.

03 *Pointer ke struct*

Pointer ke struct digunakan saat ingin bekerja dengan referensi data, terutama pada alur backend yang butuh efisiensi perubahan nilai.

Akses Field Struct

```
import (
    "fmt"
)

type User struct {
    ID    int
    Name  string
    Email string
}

func main() {
    user := User{
        ID:    101,
        Name:  "Alice",
        Email: "alice@example.com",
    }

    fmt.Println("User ID:", user.ID)
    fmt.Println("User Name:", user.Name)
```

Field struct diakses dengan operator titik. Pada pointer ke struct, Go tetap mendukung akses yang sama tanpa dereference manual.

01
10

Operator Titik

Gunakan notasi titik untuk membaca atau mengubah field pada nilai struct secara langsung.



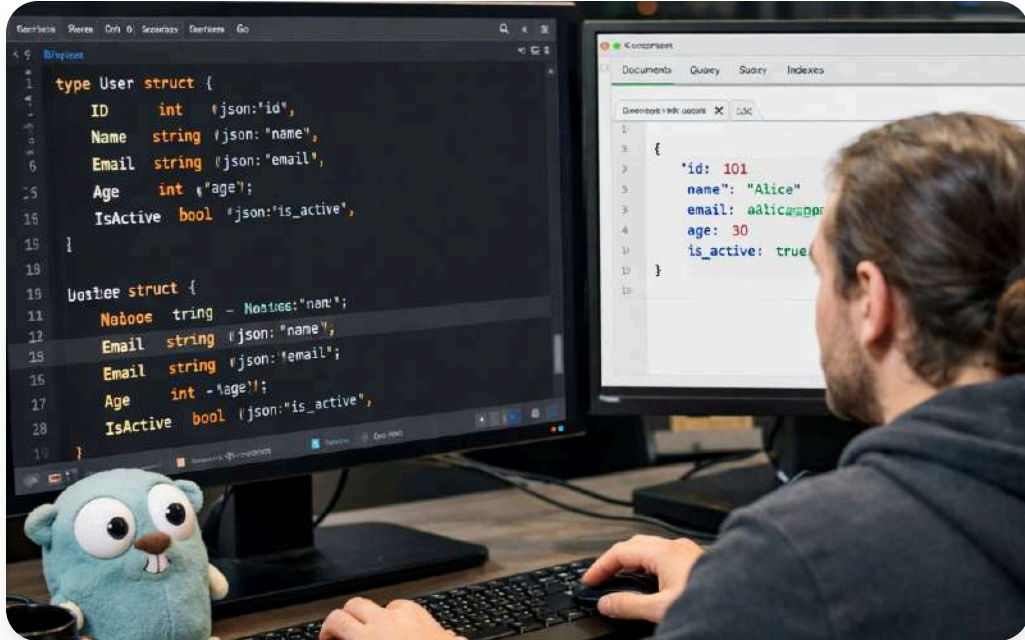
Pointer Tetap Mudah

Pointer ke struct dapat mengakses field dengan sintaks yang sama tanpa dereference manual.



Nyaman di Backend

Fitur ini memudahkan manipulasi data pada service dan handler agar kode tetap ringkas.



Pointer ke Struct

Pointer ke struct memungkinkan perubahan langsung pada objek asli, umum dipakai untuk update profil, konfigurasi, dan hasil proses backend.

Function Menerima Struct

```
}  
type Address struct {  
    City string  
    State string  
}  
  
func UpdatePerson(p Person, addr *Address) {  
    p.Name = "Updated Name"  
    p.Age += 1  
  
    addr.City = "New City"  
    addr.State = "New State"  
  
    fmt.Println("Updated Person:", p)  
}
```

01 Parameter nilai struct

Gunakan struct biasa saat function hanya membaca data dan tidak perlu mengubah isi objek.

02 Parameter pointer ke struct

Pointer cocok saat function perlu memodifikasi data asli atau mengurangi biaya copy pada struct besar.

03 Pilih sesuai kebutuhan

Gunakan nilai untuk keamanan salinan, dan pointer untuk efisiensi serta perubahan data secara langsung.



Pengenalan Method di Go

Method adalah fungsi yang terikat pada tipe, umumnya *struct*, agar data dan perilaku terkait tersusun rapi dalam kode backend.

Method dengan Value Receiver

```
import "fmt"

type Rectangle struct {
    Width float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}
```

Value receiver menerima salinan struct dan cocok untuk method yang hanya membaca data.



Menerima Salinan Nilai

Method memakai salinan struct sehingga data asli tetap aman.



Cocok untuk Operasi Baca

Digunakan saat method hanya membaca atau menampilkan isi data.



Minim Efek Samping

Pemakaiannya menandakan method tidak mengubah state object asli.

Method dengan Pointer Receiver

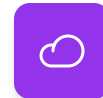


Pointer receiver menerima alamat struct agar method bisa mengubah data asli dan bekerja lebih efisien pada object besar.



Mengubah Data Asli

Method menerima alamat struct, sehingga perubahan di dalam method langsung memengaruhi object asal.



Cocok untuk Update State

Umum dipakai pada backend saat object perlu diperbarui, seperti status, cache, atau counter.



Lebih Efisien

Untuk struct besar, pointer receiver menghindari penyalinan data saat method dipanggil.

Method pada Struct

Elemen	Kode	Tujuan
Struct	User { Name, Active }	Menyimpan data pengguna
Method	func (u *User) Activate()	Menempel pada struct User
Pointer receiver	*User	Mengubah objek asli
Perubahan field	u.Active = true	Mengaktifkan status user

Contoh ini menunjukkan bahwa method **Activate** memakai *pointer receiver* agar perubahan pada field **Active** terjadi langsung pada objek **User** asli.

Value vs Pointer Receiver

01 Value Receiver

Gunakan saat method hanya membaca data, ukuran struct kecil, dan tidak perlu mengubah state objek.

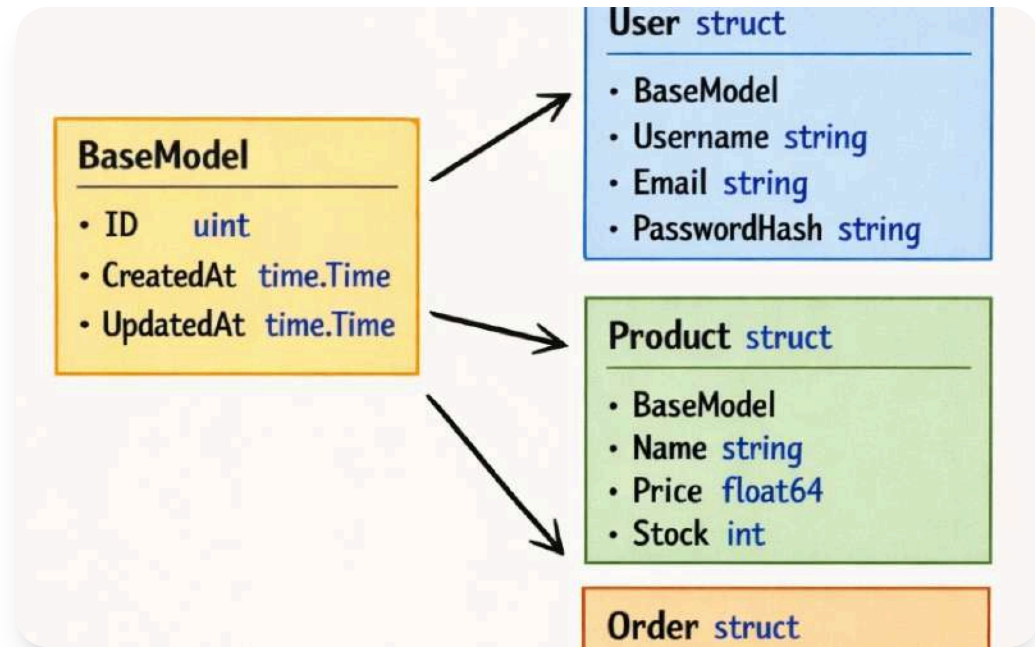
03 Konsistensi Desain

Dalam backend, gunakan pointer receiver bila ingin pola method tetap konsisten pada tipe yang sama.

02 Pointer Receiver

Pilih saat method mengubah field, struct cukup besar, atau ingin menghindari penyalinan data.

```
c.value++  
}  
  
// Pointer receiver method  
func (c *Counter) Reset() {  
    c.value = 0  
}  
  
func main() {  
    c := Counter{value: 10}  
    fmt.Println("Initial value:", c.value)  
    c.Increment()  
    fmt.Println("After Increment", c.value)
```

Embedding Struct

Go mendukung *embedding struct* untuk komposisi modular. Field dan method dari struct yang disisipkan dapat diakses dari struct luar tanpa pewarisan.

Struct dan Method Backend



Struct sebagai Komponen Utama

Dipakai untuk membentuk repository, service, handler, dan configuration object dalam aplikasi backend.



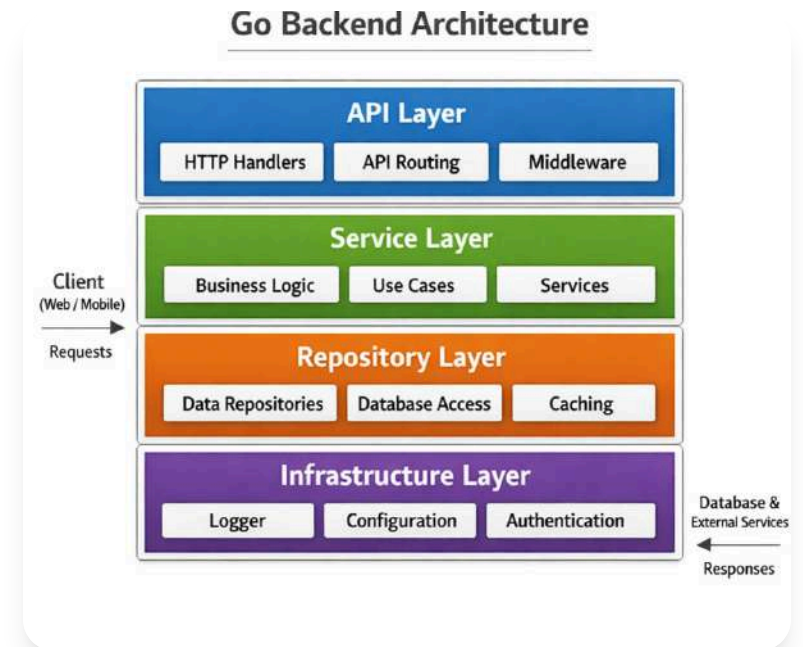
Pemisahan Tanggung Jawab

Kombinasi struct dan method membuat kode lebih mudah diuji, dipelihara, dan dikembangkan.



Method Mendefinisikan Perilaku

Method memberi fungsi pada tiap komponen agar tanggung jawabnya jelas dan alur kerja lebih terorganisir.



Kesalahan Umum yang Perlu Dihindari



Salah Memilih Receiver

Gunakan **pointer receiver** saat method perlu mengubah state atau menghindari copy data besar.



Nil Pointer dan Tanggung Jawab Berlebih

Selalu cek **nil pointer** dan hindari struct yang menampung terlalu banyak data serta logika sekaligus.



Lupa Efek Value Copy

Value receiver membuat salinan data, sehingga perubahan di method tidak memengaruhi objek asli.



Ringkasan Pertemuan



"Pointer membantu efisiensi dan perubahan data langsung, struct menjadi fondasi entitas backend, dan method memberi perilaku pada tipe data."

Materi Golang Backend